Note :
* **Static Variable, Non static variable and static methods** are always executed based on the current reference, regardless of object.

Polymorphism :

```
                    Poly = many
                    Morphs = Forms
```

* Polymorphism = **a Greek word whose meaning is "Same object having different behaviour"**.
* In our real life, a person OR a human being can perform as many task as chosen below :
  Example :
  public void person(Walking walking)
  {
  }

  public void person(Running running)
  {
  }

  public void person(Sleeping sleeping)
  {
  }

* In the same way, In our programming languages, A method OR a constructor can perform as many tasks :
  Example :

  public void add(int x, int y)
  {
  }

  public void add(double x, double y)
  {
  }

  public void add(String x, String y)
  {
  }

* Polymorphism is divided into two types :
  1) Static OR Compile time Polymorphism OR Early binding
  2) Dynamic OR Runtime Polymorphism OR Late Binding

Static Polymorphism :

* The polymorphism which **exist at the time of compilation is called static polymorphism.**

* **In static polymorphism**, Java compiler is having very good idea regarding the method invocation based on the reference variable.
* The binding of the method is done at compile time, itself so it is also known as "Early Binding".

* We can achieve static polymorphism by using "Method Overloading" concept.
  Example of Static polymorphism :
  Method Overloading

Dynamic Polymorphism :

* The polymorphism which **exist at runtime is called Dynamic Polymorphism.**

* In Dynamic polymorphism, Java compiler does not have any idea regarding method invocation, At runtime, JVM will decide which method to be invoked based on the reference variable.

* The binding of the method is done at runtime so, It is also known as Late Binding.

* We can achieve dynamic polymorphism by using "Method Overriding".

  Example of Dynamic Polymorphism :
  Method Overriding

What is final keyword in java ?

* final keyword is used to provide some kind of restriction to our java programs.

* final keyword we can use 2 ways in java.

  1) To declare a class as a final (Inheritance is not possible for final class)
  2) To declare a method as a final (Overriding is not possible)
  3) To declare a variable OR field as a final (Re-assignment is not possible)

1) To declare a class as a final :

* Whenever we declare a class as a final then we cannot extend OR inherit that class.

* If the class composition (copy of the class) is very important then we should declare the class as a final class.

* We can create the object for final class but we cannot inherits.

* String, All the Wrapper classes are final class.

//Program
final class A
{
    private int x = 100;

    public void setData(int x)
    {
        this.x = x;
        SO.println(this.x);
    }
}
class B extends A
{
}

public class FinalClass01
{
    public static void main(String[] args)
    {
        B b1 = new B();
        b1.setData(10);
    }
}

We can create an object for final class as shown below :

//Can create the object for final class.
final class Test
{
    private int data;

    public Test(int data)
    {
        this.data = data;
        SO.println("Data value is :"+data);
    }
}
public class FinalClass02
{
    public static void main(String[] args)
    {
        Test t1 = new Test(200);
    }
}

sealed keyword in java (Java 17+)

It is a new feature introduced from java 17+ (preview version) and become the integral part of java from 17th LTS.

It is an improvement over final keyword.

By using sealed keyword we can declare classes and interfaces as sealed.

It is one kind of restriction that describ that which classes and interfaces can extend or implement from Sealed class OR interface.

It is similar to Final keyword with less restriction because here we can permit the classes to extend from the original Sealed class.

The class which is inherited from the sealed class must be final, sealed or non-sealed.

The sealed class must have atleast one sub class.

We can create object for Sealed class.

It provides the following modifiers :

1) sealed : Can be extended only through permitted class.

2) non-sealed : Can be extended by any sub class, if a user needs to give permission to its sub classes.

3) permits : We can provide permission to the sub classes, which are inherited through Sealed class OR sealed interface.

4) final : we can declare permitted sub class as final so, it cannot be extended further.

//Program :
package com.first.sealed;
sealed class Bird permits Parrot, Sparrow
{
    public void fly();
    {
        SO.println("Generic Bird is flying");
    }
}
non-sealed class Parrot extends Bird
{
    public void fly();
    {
        SO.println("Parrot Bird is flying");
    }
}
final class Sparrow extends Bird
{
    public void fly();
    {
        SO.println("Sparrow Bird is flying");
    }
}

public class SealedDemo
{
    public static void main(String[] args)
    {
        Bird bird = null;
        bird = new Parrot(), bird.fly();
        bird = new Sparrow(), bird.fly();
    }
}

---

2) To declare a method as a final (Overriding is not possible)

Whenever we declare a method as a final then we can't override that method in the sub class otherwise there will be a compilation error.

We should declare a method as a final if the body of the method i.e the implementation of the method is very important and we don't want to change it in the future. Super class method body to sub class method body flow we should declare the super class method as final method.

//Program :
class A
{
    protected int x = 10;
    protected int y = 20;

    public final void calculate()
    {
        int sum = x+y;
        SO.println("Sum is :"+sum);
    }
}
class B extends A
{
    @Override
    public void calculate() //ERROR
    {
        int mul = x*y;
        System.out.println("Mul is :"+mul);
    }
}

public class FinalMethod02x
{
    public static void main(String [] args)
    {
        B b1 = new B();
        b1.calculate();
    }
}

class Alpha
{
    private final void accept()
    {
        System.out.println("Alpha class accept method");
    }
}
class Beta extends Alpha
{
    public void accept()
    {
        System.out.println("Beta class accept method");
    }
}

public class FinalMethod03x
{
    public static void main(String [] args)
    {
        new Beta().accept();
    }
}

Note : private method of a super class is not not available to sub class so the above code will compile

---

3) To declare a variable/Field as a final :

In other languages like C and C++ we use "const" keyword to declare a constant variable but in java, const is a reserved keyword but not used. So we should use "final" keyword.

If we declare a variable/field as a final then we can't perform re-assignment (i.e editing but re-initialization) of that variable.

class A
{
    final int X = 20;

    public void setVal(x)
    {
        X = 10; //error
        System.out.println(X);
    }
}
class FinalVar3x
{
    public static void main(String [] args)
    {
        A a1 = new A();
        a1.setVal();
    }

}

class Student
{
    private final int rollNumber;
    private final String studentName;

    public Student(final int rollNumber, Final String studentName)
    {
        this.rollNumber = rollNumber;
        this.studentName = studentName;
    }

    @Override
    public String toString()
    {
        return "Roll is :"+this.rollNumber+" Name is :"+this.studentName;
    }
}
class FinalVar3x1
{
    public static void main(String[] args)
    {
        final Student s = new Student(11, "Sunil");
        System.out.println(s);

        s1 = new Student(22,"Ram"); **//illegalAS //error**
        System.out.println(s1);
    }
}

Assignment :

* **How to create immutable classes in java (String & all the Wrapper classes are immutable)**

Offline :

1) Abstraction (Serious Polymorphism)
2) Interface (Loose Coupling)
3) Static OR Final Methods
4) Exception Handling
5) Java 8 features (Lambda)
6) Multithreading (Video)
7) Multithreading + Concurrent Collection (Video)

Sunday Evening Topic (Online)

1) Inner classes in Java
2) Object class & its methods
3) Wrapper classes (Boxing & Unboxing)
4) Generics and wildcards
5) Collection Framework

What is abstraction in java ? [Serious Polymorphism]

* Showing the **essential details** without showing the **background details** (Implementation part) is called abstraction.
  Example :
  ATM Machine, Keyboard, Remote Control, Laptop and so on

* We can achieve abstraction by using the following 2 concepts in java :

  1) **Abstract class & abstract methods** [0 to 100% abstraction i.e Partial Abstraction]
  2) **Interface** [100 % abstraction] [Full Abstraction]

What is an abstract method in java :

* In Java, We have a special method which does not provide any **method body OR un-implemented method (Method Without any implementation) is called abstract method.**

  Example :
  public abstract void draw();

* In order to define abstract method, we should compulsory use abstract keyword.